

# Curs 6 — Structuri de Date (Gabriel Istrate)

## Sinteză: Arbori auto-balansați — AVL, Splay, Red-Black

🎯 = punct de examen, ⚠️ = capcană, ★ = fapt fundamental, 🌱 = gol completat.

Cursul acoperă trei arbori auto-balansați care, în sfârșit, ating bariera  $O(\log n)$ . E unul dintre cele mai testate cursuri — AVL și Red-Black apar pe toate grilele.

### 0. De ce e critic de examen

| Tip de întrebare                              | Unde apare                            | Ce-ți trebuie                      |
|-----------------------------------------------|---------------------------------------|------------------------------------|
| AVL: noduri max / înălțime min-max            | grila2025 Q1, grila2023 Q1, model Q1  | definiția balansării AVL           |
| AVL: care nod e ultimul inserat (din BF)      | grila2025 Q9                          | balance factor $\in \{-1, 0, +1\}$ |
| AVL: câte reprezentări                        | grila2025 Q20, model Q19              | numeri formele valide              |
| AVL: adevărat/fals (rotații simple+double)    | grila2023 Q4                          | ambele restabilesc invariantul     |
| Red-Black: ce noduri sigur roșii/negre        | grila2025 Q7, grila2023 Q8, model Q25 | cele 5 proprietăți + black-height  |
| Red-Black: câte reprezentări                  | model Q18, grila2023 Q18              | proprietățile + numărare           |
| Red-Black: desenează după inserări            | toamnă Q5                             | RB-Insert + fixup                  |
| Splay: proprietăți / garanții                 | grila2023 Q14, grila2025 Q17          | <b>amortizat ≠ worst-case</b>      |
| Care arbori garantează $O(\log n)$ worst-case | grila2025 Q17                         | AVL/RB <b>DA</b> , Splay <b>NU</b> |
| În care arbori facem rotații                  | grila2025 Q19                         | AVL, splay, red-black — toate      |

## 1. Recap BST + motivația auto-balansării

**Rezumat BST:** întruchipează strategia divide-and-conquer; Search/Insert/Min/Max sunt  $O(h)$ ; în general  $h(n) = \Omega(\log n)$  și  $h(n) = O(n)$ ; randomizarea face cazul defavorabil  $h(n)=n$  foarte improbabil.

⚠ **Problema:** cazul defavorabil e improbabil dar **tot posibil**, iar cazurile „pur și simplu proaste” (input parțial sortat) sunt și mai probabile.

**Ideea auto-balansării:** dacă o inserare/ștergere dezechilibrează arborele, îl **reechilibrezi**. Pentru căutări frecvente merită — listă = căutare liniară, arbore echilibrat = căutare logaritmică. Avantajul ține cât timp arborele rămâne „aproximativ echilibrat”.

## 2. De ce trei? (comparație AVL vs Red-Black vs Splay)

| Criteriu                | AVL                                  | Red-Black                     | Splay                                              |
|-------------------------|--------------------------------------|-------------------------------|----------------------------------------------------|
| Echilibrare             | mai strict echilibrat                | mai puțin strict              | nu garantează strict                               |
| Bun pentru              | căutări (lookup)                     | inserări                      | acces neuniform (puține noduri „fierbinți”)        |
| Complexitate            | $O(\log n)$ <b>worst-case</b>        | $O(\log n)$ <b>worst-case</b> | $O(\log n)$ <b>amortizat</b>                       |
| Simplitate implementare | cea mai grea                         | medie                         | cea mai simplă                                     |
| Memorie                 | stochează info de balanță            | stochează culoarea            | <b>nu stochează info de balanță</b> → mai eficient |
| Concurență              | bun multithreaded (căutări paralele) | —                             | —                                                  |
| Benchmark               | ~20% mai rapid ca RB la lookups      | —                             | prost la acces în ordine sortată                   |

**În practică:** Red-Black în Java (`TreeMap`, `TreeSet`), C++ STL (`map`, `multimap`, `multiset`), kernel Linux (Completely Fair Scheduler). Splay în cache-uri, alocatoare de memorie, routere, garbage collectors, compresie. (Alți arbori: treaps, T-trees, tango trees.)

### 3. AVL Trees (Adelson-Velskii și Landis)

#### 3.1 Condiția de echilibru

Cursul construiește definiția prin eliminare:

- **Condiția #1:** subarborele stâng și drept ai **rădăcinii** au aceeași înălțime → **prea slabă**.
- **Condiția #2:** subarborele stâng și drept ai **fiecărui** nod au aceeași înălțime → **prea tare**.
- ★ **Condiția AVL (#3):** pentru fiecare nod  $v$ , subarborii stâng și drept ai lui  $v$  au înălțimi care **diferă cu cel mult 1**.

#### 3.2 ★ Balance factor (factorul de echilibru)

**Balance factor** al unui nod = **diferența de înălțime între subarborele stâng și cel drept** =  $h(\text{stânga}) - h(\text{dreapta})$ . Într-un AVL, fiecare nod are balance factor **0 sau  $\pm 1$** .

🌀 Asta e definiția testată la grila2025 Q9 („care nod poate fi ultimul inserat?”) — analizezi factorii de balanță.

#### 3.3 Reparația prin rotații

Când inserarea încalcă condiția AVL, se face o reparație, **chiar la momentul încălcării**, prin **rotații simple** sau **duble**.

★ **Faptul cheie:** după o rotație (simplă sau dublă), noua înălțime a întregului subarbor e **exact aceeași** cu înălțimea subarborului original dinaintea inserării care a cauzat dezechilibrul. → **Nu mai trebuie actualizate înălțimile pe drumul spre rădăcină și nu mai sunt necesare alte rotații**. De-asta inserarea AVL face  $O(1)$  rotații.

#### 3.4 🌀 Care rotație folosești (regula de recunoaștere)

„Recunoașterea rotației corecte e partea cea mai grea.” Algoritmul cursului:

1. Găsește **nodul dezechilibrat** (primul nod, urcând de la inserare, cu  $|\text{balance factor}| = 2$ ).
2. Coboară **două noduri** către nodul nou inserat.
3. Dacă drumul e **drept** → **rotație simplă**.
4. Dacă drumul **zigzag-uiește** → **rotație dublă**.

Cele patru cazuri concrete:

| Caz | Unde s-a inserat                     | Rotație                |
|-----|--------------------------------------|------------------------|
| LL  | subarborele stâng al copilului stâng | simplă la dreapta      |
| RR  | subarborele drept al copilului drept | simplă la stânga       |
| LR  | subarborele drept al copilului stâng | dublă (stânga-dreapta) |
| RL  | subarborele stâng al copilului drept | dublă (dreapta-stânga) |

🌀 De aici răspunsul la grila2023 Q4: **și rotațiile simple, și cele duble** sunt folosite pentru a restabili invariantul AVL (variantele (b) și (c) ambele adevărate).

### 3.5 Ștergere în AVL

- Folosești `deleteByCopying()` — reduce ștergerea unui nod cu doi descendenți la ștergerea unui nod cu cel mult un descendent (delete by copy, ca la BST).
- După ștergere, **actualizezi factorii de balanță de la părintele nodului șters până la rădăcină.**
- Pentru fiecare nod al cărui balance factor devine  $\pm 2$ , faci o rotație simplă sau dublă ca să restabilești echilibrul.
- ⚠ Spre deosebire de inserare ( $O(1)$  rotații), **ștergerea poate cere până la  $O(\log n)$  rotații** — o ștergere poate îmbunătăți balance factor-ul părintelui, dar îl poate înrăutăți pe al bunicului, deci efectul se propagă în sus.

## 4. Splay Trees (Sleator și Tarjan, 1985)

„To splay” ~ a împrăștia. Arbore auto-balansat, **mai simplu de implementat** decât AVL/RB, cu o proprietate în plus: **elementele accesate recent sunt rapid de accesat din nou.**

### 4.1 ★ Complexitate

Inserare, căutare, ștergere:  **$O(\log n)$  amortizat**. Asta înseamnă, aproximativ, că **prețul mediu per operație într-o secvență lungă de operații e  $O(\log n)$**  — dar o operație individuală poate fi mai scumpă.

🌀 Asta e distincția testată: splay-ul NU garantează  $O(\log n)$  în cazul cel mai rău (ca AVL/RB), ci doar **amortizat**. La grila2025 Q17 („care arbori garantează  $O(\log n)$  worst-case”), splay-ul **NU** intră în listă.

## 4.2 Operația fundamentală: splaying

Când un nod  $x$  e accesat, se face o operație de **splaying** care îl aduce în vârf, printr-o secvență de **pași de splaying**; fiecare pas îl apropie pe  $x$  de rădăcină. Pasul depinde de: dacă  $x$  e copil stâng/drept al părintelui  $p$ ; dacă  $p$  e rădăcină; dacă  $p$  e copil stâng/drept al bunicului  $g$ . **Trei tipuri de pași**:

1. **zig** ( $p$  e rădăcina): practic o rotație.
2. **zig-zig** ( $x$  și  $p$  ambii copii stângi sau ambii copii dreپți).
3. **zig-zag** ( $x$  și  $p$  alternează părțile). (Fiecare caz are de fapt două variante în oglindă.)

## 4.3 Avantaje / dezavantaje

- ✓ Nodurile mai des accesate ajung mai aproape de rădăcină — util pentru cache-uri, garbage collection.
- ✓ Mai eficient ca memorie decât AVL (nu stochează informație de balanță în noduri).
- ✗ Accesul aleator e mai prost decât la alți BST echilibrați.
- ⚠ **Foarte prost: accesul elementelor în ordine sortată** (cazul defavorabil pentru splay).

🔗 La grila2023 Q14 („ce e adevărat într-un splay tree”), răspunsul e „niciun alt răspuns” — splay-ul NU e height-balanced (ca AVL), NU are noduri colorate (ca RB), subarborii rădăcinii NU au aceeași înălțime.

---

## 5. Red-Black Trees

### 5.1 ★ Cele 5 proprietăți (de memorat exact)

1. Fiecare nod e roșu sau negru.
2. Rădăcina e neagră.
3. Fiecare frunză (NIL) e neagră.
4. Dacă un nod e roșu, ambii copii ai săi sunt negri (nu există două noduri roșii adiacente).
5. Pentru fiecare nod  $x$ , fiecare drum de la  $x$  la frunzele descendente are același număr de noduri negre = black-height  $\boxed{bh(x)}$ .

### 5.2 Implementare

- Un nod santinelă comun ( $\boxed{T.nil}$ ) reprezintă toate frunzele NIL; santinela e și părintele rădăcinii.

- $T.root$ ,  $T.nil$ ; fiecare nod are  $parent$ ,  $key$ ,  $left$ ,  $right$ , și  $color \in \{roșu, negru\}$ .

### 5.3 ★ Înălțimea: lema fundamentală

**Lemă:** înălțimea  $h(x)$  a unui red-black tree cu  $n$  noduri interne e cel mult  $2 \cdot \log(n+1)$ .

**Demonstrație** (utilă la Nivel II):

1. **Sublema:**  $size(x) \geq 2^{bh(x)} - 1$ , prin inducție.
  - Bază:  $x$  e frunză  $\rightarrow size\ 0$ ,  $bh\ 0$ , iar  $2^0 - 1 = 0 \checkmark$ .
  - Pas: copiii lui  $x$  au  $bh \geq bh(x) - 1$  (black-height-ul unui copil e  $bh(x)$  dacă e roșu,  $bh(x)-1$  dacă e negru — oricum  $\geq bh(x)-1$ ). Din ipoteză, fiecare subarbore-copil are  $\geq 2^{bh(x)-1} - 1$  noduri. Deci  $size(x) = size(y_1) + size(y_2) + 1 \geq 2 \cdot (2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1 \checkmark$ .
2. Fiindcă orice nod roșu are copii negri (proprietatea 4), pe orice drum de la  $x$  la o frunză **cel puțin jumătate** din noduri sunt negre  $\rightarrow bh(x) \geq h(x)/2$ .
3. Combinând:  $n = size(x) \geq 2^{bh(x)} - 1 \geq 2^{h(x)/2} - 1$ , deci  $2^{h/2} \leq n+1$ , adică  $h \leq 2 \cdot \log(n+1)$ .

🔗 **Consecința crucială:** un red-black tree lucrează ca un BST pentru căutare etc., iar  $T(n) = O(h) = O(\log n)$  — **și asta e complexitatea în cazul cel mai rău**. De-asta RB (și AVL) garantează  $O(\log n)$  worst-case, iar grila2025 Q17 le include.

### 5.4 RB-Insert (inserare)

Funcționează **ca inserarea într-un BST**, dar trebuie să **păstreze proprietățile red-black**. Strategia generală:

1. Inserezi  $z$  ca într-un BST (ca frunză).
2. **Colorezi  $z$  roșu** (ca să păstrezi proprietatea 5 — black-height-ul nu se schimbă dacă noul nod e roșu).
3. Repari arborele ca să corectezi posibile încălcări ale **proprietății 4** ( $z$  roșu cu părinte roșu).

```
RB-Insert(T, z):
    ... (inserare BST normală) ...
    z.left = z.right = T.nil
    z.color = red
    RB-Insert-Fixup(T, z)
```

## 5.5 RB-Insert-Fixup — cazurile („desenează arborele")

După ce z e inserat roșu, singura problemă posibilă e roșu-roșu (z și părintele roșii). Te uiți la **unchiul** lui z (fratele părintelui):

- **Părintele lui z e negru** → **niciun fixup necesar** (totul e ok).
- **Caz 1 — unchiul lui z e ROȘU: recolorezi** — bunicul (negru) devine roșu și își „transferă" negrul celor doi copii (părinte și unchi, ambii devin negri). Asta poate crea alte conflicte roșu-roșu mai sus, deci **iterezi în sus**. La final, rădăcina poate fi colorată negru fără probleme.
- **Caz 3 — unchiul e NEGRU, conflict „în linie" (zig-zig):** se rezolvă cu o rotație plus o schimbare de culoare.
- **Caz 2 — unchiul e NEGRU, conflict „zig-zag":** mai întâi o rotație ca să-l transformi într-un conflict în linie, apoi rotație + schimbare de culoare (cazul 3).

(Reține tiparul: **unchi roșu** → recolorezi și urci; **unchi negru** → rotești și recolorezi.)

## 5.6 RB-Delete (ștergere)

Pornește de la ștergerea BST (cele 3 cazuri: 0 copii / 1 copil / 2 copii → succesor). Apoi:

- **Ștergerea unei frunze roșii** → **niciun ajustaj** (nu afectează black-height-ul, nu creează roșu-roșu adiacent).
- **Ștergerea unui nod negru** → strică echilibrul de black-height → **RB-Delete-Fixup**.

**Concept cheie — „greutatea neagră în plus" (extra black weight):** când scoți un nod negru, drumurile prin acel loc pierd un negru. Modelăm asta zicând că nodul înlocuitor x **poartă un negru în plus** (e „dublu negru"). Algoritmul de fixup **împinge acest negru în plus în sus** spre rădăcină, până când:

- ajunge la **rădăcină** → îl arunci (gata);
- ajunge la un **nod roșu** → nodul roșu **absoarbe** negrul în plus (devine negru);
- altfel → se rezolvă cu **rotații + transferuri de culoare** care păstrează toate celelalte proprietăți RB.

**Condițiile de fixup** (y = nodul scos efectiv, x = unicul copil al lui y sau T.nil):

- dacă **y e roșu** → niciun fixup necesar; deci presupunem y negru.
-  **Problem 1:** y = rădăcină și x roșu → încalcă **proprietatea 2** (slide-ul are „proprietatea ??” — completarea corectă e **proprietatea 2: rădăcina trebuie să fie neagră**).
- **Problem 2:** și x, și y.parent sunt roșii → încalcă **proprietatea 4** (fără roșii adiacente).
- **Problem 3:** scoatem y, care e negru → încalcă **proprietatea 5** (black-height egal).

`RB-Delete-Fixup` are **4 cazuri** (diagramele B-A-D-C-E), tratate într-o buclă `while x ≠ T.root and x.color = black`, în funcție de culoarea fratelui w al lui x și a copiilor lui. ⚠

Pentru grilă (Nivel I) NU trebuie să memorezi cele 4 cazuri pe de rost — îți trebuie conceptul (negru în plus împins în sus) și proprietățile. Cazurile detaliate sunt mai degrabă Nivel II / „desenează”.

## 6. Cheat-sheet — fapte de examen

| Concept                         | De reținut                                                                                        |
|---------------------------------|---------------------------------------------------------------------------------------------------|
| AVL — condiție                  | înălțimile subarborilor fiecărui nod diferă cu $\leq 1$                                           |
| AVL — balance factor            | $h(\text{stânga}) - h(\text{dreapta}) \in \{-1, 0, +1\}$                                          |
| AVL — care rotație              | drum drept $\rightarrow$ simplă; zig-zag $\rightarrow$ dublă                                      |
| AVL — cazuri                    | LL $\rightarrow$ dreapta, RR $\rightarrow$ stânga, LR $\rightarrow$ dublă, RL $\rightarrow$ dublă |
| AVL — rotația restaurează       | înălțimea originală $\rightarrow$ fără cascadă                                                    |
| AVL — nr. rotații               | inserare $O(1)$ , ștergere $O(\log n)$                                                            |
| Splay — complexitate            | $O(\log n)$ AMORTIZAT (nu worst-case)                                                             |
| Splay — pași                    | zig, zig-zig, zig-zag                                                                             |
| Splay — memorie                 | nu stochează info de balanță                                                                      |
| Splay — punct slab              | acces în ordine <b>sortată</b>                                                                    |
| RB — 5 proprietăți              | roșu/negru; rădăcină neagră; frunze NIL negre; roșu $\rightarrow$ copii negri; black-height egal  |
| RB — înălțime                   | $h \leq 2 \cdot \log(n+1) \rightarrow O(\log n)$ worst-case                                       |
| RB — inserare                   | inserezi ca BST, colorezi <b>roșu</b> , repari proprietatea 4                                     |
| RB — fixup inserare             | unchi roșu $\rightarrow$ recolorezi+urci; unchi negru $\rightarrow$ rotești+recolorezi            |
| RB — ștergere                   | frunză roșie trivial; nod negru $\rightarrow$ „negru în plus” împins în sus                       |
| $O(\log n)$ worst-case garantat | <b>AVL, RB DA</b> ; Splay NU (amortizat)                                                          |



| Concept                   | De reținut              |
|---------------------------|-------------------------|
| Rotații se fac în         | AVL, splay, red-black   |
| Mai strict echilibrat     | AVL > Red-Black         |
| Mai simplu de implementat | Splay > Red-Black > AVL |

## 7. Conexiuni de examen, pas cu pas

**AVL — noduri/înălțime** (grila2025 Q1, grila2023 Q1): folosești definiția balansării. Noduri max la înălțime  $h = 2^{(h+1)} - 1$ ; noduri min  $N(h) = N(h-1) + N(h-2) + 1 \rightarrow 1, 2, 4, 7, 12$ . (Detaliile numerice sunt în harta de examene.)

**AVL — ultimul nod inserat** (grila2025 Q9): nodul nou e o frunză; analizezi balance factor-ii și cauți frunza a cărei adăugare produce exact configurația dată (fără rotații).

**AVL — adevărat/fals** (grila2023 Q4): și rotații simple și duble restabilesc invariantul  $\rightarrow$  (b) și (c).

**Red-Black — ce noduri sigur negre/roșii** (grila2025 Q7, grila2023 Q8): folosești proprietățile — rădăcina e neagră (prop. 2); deduci restul din black-height-ul egal (prop. 5) și interdicția roșu-roșu (prop. 4). Un nod e forțat roșu/negru când alternativa ar strica una din proprietăți.

**Red-Black — câte reprezentări** (model Q18, grila2023 Q18):  $\{1, 2, 3\} \rightarrow 2$  moduri (totul negru echilibrat / rădăcină neagră cu doi copii roșii), respectând cele 5 proprietăți.

**Splay** (grila2023 Q14, grila2025 Q17): NU e echilibrat în înălțime, NU e colorat, dă doar  $O(\log n)$  **amortizat** — deci nu intră în lista garanțiilor worst-case.

---

*Bottom line: din acest curs trebuie să ieși cu (1) AVL — balance factor  $\in \{-1, 0, +1\}$ , drum drept  $\rightarrow$  rotație simplă / zig-zag  $\rightarrow$  rotație dublă, ambele restaurează înălțimea (deci inserare  $O(1)$  rotații, ștergere  $O(\log n)$ ); (2) Red-Black — cele 5 proprietăți pe de rost, înălțimea  $\leq 2 \log(n+1)$  care dă  $O(\log n)$  worst-case, inserarea colorează roșu și repară proprietatea 4 (unchi roșu  $\rightarrow$  recolorezi, unchi negru  $\rightarrow$  rotești); (3) Splay —  $O(\log n)$  DOAR amortizat, prost la acces sortat, nu stochează balanță. Cheia transversală: AVL și Red-Black garantează  $O(\log n)$  în cazul cel mai rău, Splay doar amortizat — exact ce se cere la întrebarea cu garanțiile. Detaliile de RB-Delete-Fixup (cele 4 cazuri) sunt Nivel II; pentru grilă îți ajunge conceptul de „negru în plus”.*